Some exercises on properties of predicate transformers

Consider the following simplified programming language with only assignments and sequential composition. Programs of this language only have access to two registers/variables: p and q, both stores boolean values.

$$
\begin{array}{lcll}
\mathit{Stmt} & ::= & \mathit{variable} := \mathit{Expr} & \text{, assignment} \\
 & | & \mathit{Stmt}\ ;\ \mathit{Stmt} & \text{, sequential composition} \\
 & & & \\
\mathit{Expr} & ::= & \mathit{variable} & \\
 & | & \mathit{constant} & \text{, "true" or "false"} \\
 & | & \neg \mathit{Expr} & \\
 & | & \mathit{Expr} \wedge \mathit{Expr} & \\
 & & & \\
\mathit{variable} & ::= & \text{"}p\text{"} \mid \text{"}q\text{"} &
\end{array}
$$

Consider the semantics of statements and Hoare triple shown below —these are similar to the semantics discussed in the course slides. To keep the notation simpler, we will just write, for example, $[\![expr]\!]$ and $[\![strmt]\!]$ instead of the full out $\mathcal{S}[\![stmt]\!]$ and $\mathcal{E}[\![expr]\!]$.

We will define the semantic of a statement as a function of type $state \to state$; the semantic of an expression is a functionof type $state \to value$, and the semantic of a predicate (boolean expression) as a function of type $state \to \mathsf{bool}$.

If F is a function, applying it on an argument x is denoted as $F\ x$ (a la Haskell), or $F(x)$, if that would make it clearer.

Because we only have two variables, p and q, a state s will be represented as a pair $s = \langle p : value_1, q : value_2 \rangle$. We use the notation $s.x$ and $s.y$ to get the value of the corresponding element.

The semantics of expressions:

$$
\begin{array}{lcll}
[\![v]\!] & = & (\lambda s \bullet\ s.v) & \text{, where } v \text{ is a variable} \\
[\![c]\!] & = & (\lambda s \bullet\ c) & \text{, where } c \text{ is a constant} \\
[\![\neg expr]\!] & = & (\lambda s \bullet\ \neg([\![expr]\!]s)) & \\
[\![\neg expr_1 \wedge expr_2]\!] & = & (\lambda s \bullet\ [\![expr_1]\!]s\ \wedge\ [\![expr_2]\!]s) &
\end{array}
$$

The semantics of statements and Hoare triple:

$$
\begin{array}{lcll}
\mathcal{S}[\![\text{"}p\text{"} := expr]\!] & = & (\lambda s \bullet\ \langle p{:}value,\ q{:}s.q \rangle) & \text{, where } value = [\![expr]\!]s \\
\mathcal{S}[\![\text{"}q\text{"} := expr]\!] & = & (\lambda s \bullet\ \langle p{:}s.p,\ q{:}value \rangle) & \text{, where } value = [\![expr]\!]s \\
\mathcal{S}[\![S_1; S_2]\!] & = & (\lambda s \bullet\ [\![S_2]\!]\ ([\![S_1]\!]\ s)) &
\end{array}
$$

$$
[\![\{P\}\ stmt\ \{Q\}]\!] \quad = \quad (\forall s :: [\![P]\!]\ s\ \Rightarrow\ [\![Q]\!]([\![stmt]\!]\ s))
$$

Definition of wlp is as usual:

$$
\begin{array}{lcl}
\mathsf{wlp}\ \text{"}p\text{"} := expr\ Q & = & Q[expr/p] \\
\mathsf{wlp}\ \text{"}q\text{"} := expr\ Q & = & Q[expr/q] \\
\mathsf{wlp}\ (stmt_1; stmt2)\ Q & = & \mathsf{wlp}\ stmt_1\ (\mathsf{wlp}\ stmt_2\ Q)
\end{array}
$$

Substitution e/v is also defined as usual, but then limited to the syntax of expression of the above programming language.

$$
\begin{array}{lcll}
c[e/v] & = & c & , v \text{ is either "p" or "q"} \\
"p"[e/p] & = & e & \\
"p"[e/q] & = & p & \\
"q"[e/p] & = & q & \\
"q"[e/q] & = & e & \\
(\neg expr)[e/v] & = & \neg(expr[e/v]) & , v \text{ is either "p" or "q"} \\
(expr_1 \wedge expr_2)[e/v] & = & expr_1[e/v] \wedge expr_2[e/v] & , v \text{ is either "p" or "q"}
\end{array}
$$

1. wlp is **sound and complete** if for any post-condition Q, for any statement $stmt$, and for any state s, we have:

   s satisfies wlp $stmt$ Q if and only if the state $t = [\![stmt]\!]s$ satisfies the post-condition Q.

   This can be written more formally as the following equivalence:

   $$[\![\mathsf{wlp}\ stmt\ Q]\!]\ s\ \equiv\ [\![Q]\!]\ ([\![stmt]\!]s)$$

   Prove that the above wlp is sound and complete over assignments.

   Proof

   We want to prove that the following hold, for any post-condition Q and any state s:

   $$[\![Q[expr/v]]\!]s\ \equiv\ [\![Q]\!]([\![v := expr]\!]s)$$

   Because in this programming language we only have p and q as variables to store values, consequently we only ony have two kinds of assignment: "p" $:=$ $expr$ or "q" $:=$ $expr$ (this was also made explicit in the syntax).

   Let consider the assignment $p := expr$. The case for the other assignment, $q := expr$ is analogous. So, we want to prove this equivalence for every state s:

   $$[\![Q[expr/p]]\!]s\ \equiv\ [\![Q]\!]\ (\underbrace{[\![p := expr]\!]s}_{t})$$

   We'll prove this by induction over the structure of the post-condition Q.

   (2a) Case that Q is a constant c.

   $[\![Q[expr/p]]\!]s$
   $=$ Q is a constant c
   $[\![c[expr/p]]\!]s$
   $=$
   $[\![c]\!]s$
   $=$ well ... c is a constant, so if it holds on s, it holds on any t
   $[\![c]\!]\ \underbrace{([\![p := expr]\!]s)}_{t}$

   (2b) Case that Q is the variable "p".

$[\![Q[expr/p]]\!]s$
$=$ Q is just "p"
$[\![p[expr/p]]\!]s$
$=$ applying the substitution
$[\![expr]\!]s$
$=$
$[\!["p"]\!]\ \langle p{:}[\![expr]\!]s,\ q{:}s.q\rangle$
$=$
$[\!["p"]\!]\ \underbrace{([\![p := expr]\!]s)}_{t}$

(2c) Case that Q is the variable "q".

$[\![Q[expr/p]]\!]s$
$=$ Q is just "q"
$[\![q[expr/p]]\!]s$
$=$ applying the substitution
$[\!["q"]\!]s$
$=$
$[\!["q"]\!]\ \langle p{:}[\![expr]\!]s,\ q{:}s.q\rangle$
$=$
$[\!["q"]\!]\ \underbrace{([\![p := expr]\!]s)}_{t}$

(2d) Case that Q is of the form $\neg E$.

$[\![Q[expr/p]]\!]s$
$=$ Q is of the form $\neg E$
$[\![(\neg E)[expr/p]]\!]s$
$=$ applying the substitution
$[\![\neg(E[expr/p])]\!]s$
$=$ applying the semantic of $\neg$
$\neg([\![E[expr/p]]\!]s)$
$=$ applying induction hypothesis
$\neg\ ([\![E]\!]\ \underbrace{([\![p := expr]\!]s)}_{t})$
$=$ applying the semantic of $\neg$ in backwards direction
$[\![\neg E]\!]\ \underbrace{([\![p := expr]\!]s)}_{t}$

(2d) Case that Q is of the form $E \wedge F$.

$[\![Q[expr/p]]\!]s$
$=$ Q is of the form $E \wedge F$
$[\![(E \wedge F)[expr/p]]\!]s$
$=$ applying the substitution
$[\![E[expr/p] \wedge F[expr/p]]\!]s$
$=$ applying the semantic of $\wedge$
$([\![E[expr/p]]\!]s)\ \wedge\ ([\![F[expr/p]]\!]s)$
$=$ applying induction hypothesis
$([\![E]\!]\ \underbrace{([\![p := expr]\!]s)}_{t})\ \wedge\ ([\![F]\!]\ \underbrace{([\![p := expr]\!]s)}_{t})$
$=$ applying the semantic of $\wedge$ in backwards direction
$[\![E \wedge F]\!]\ \underbrace{([\![p := expr]\!]s)}_{t}$

2. Assuming that wlp is sound and complete over $stmt_1$ and $stmt_2$, prove that it is also sound and complete over $stmt_1; stmt_2$.

Proof

$[\![\mathsf{wlp}\ (stmt_1\ ; stmt_2)\ Q]\!]\ s$
= def. wlp
$[\![\mathsf{wlp}\ stmt_1\ (\underbrace{\mathsf{wlp}\ stmt_2\ Q}_{R})]\!]\ s$
= applying induction hypothesis on $stmt_1$
$[\![\underbrace{\mathsf{wlp}\ stmt_2\ Q}_{R}]\!]\ (\underbrace{[\![stmt_1]\!]s}_{t})$
= applying induction hypothesis on $stmt_2$
$[\![stmt_2]\!]\ (\underbrace{([\![stmt_1]\!]s)}_{t})$
- applying the semantic of ";", in backwards direction
$[\![stmt_1; stmt_2]\!]\ s$

3. Prove that wlp is sound and complete over any statement in the above given programming language.

Solution: we can prove this via induction over the structure of *stmt*. In no-1 we have proven the base case, namely assignment. No-2 is essentially the proof of the induction case.

4. Prove that wlp is monotonic. That is if $P \Rightarrow Q$ is valid, then the implication $\mathsf{wlp}\ stmt\ P\ \Rightarrow\ \mathsf{wlp}\ stmt\ Q$ is also valid.

Proof:

So, for any state s, if s satisfies $\mathsf{wlp}\ stmt\ P$, we have to prove that s also satisfies $\mathsf{wlp}\ stmt\ Q$. We will make use of the previous result, where you have proven that wlp is sound and complete over our small language of statements.

s satisfies $\mathsf{wlp}\ stmt\ P$
= wlp is sound and complete
state $t = [\![stmt]\!]\ s$ satisfies P
= $P \Rightarrow Q$
state $t = [\![stmt]\!]\ s$ also satisfies Q
= wlp is sound and complete
s satisfies $\mathsf{wlp}\ stmt\ Q$

Done.

5. Prove that wlp is distributive over $\wedge$. So, prove that this hold for any statement *stmt* and post-conditions P and Q:

$$\mathsf{wlp}\ stmt\ (P \wedge Q)\ =\ \mathsf{wlp}\ stmt\ P\ \wedge\ \mathsf{wlp}\ stmt\ Q$$

Proof: we prove this by induction over the structure of *stmt*.

(1) For assignment:

$\mathsf{wlp}\ (v := expr)\ (P \wedge Q)$
$=$ def. wlp
$(P \wedge Q)[expr/v]$
$=$ def. substitution
$P[expr/v]\ \wedge\ Q[expr/v]$
$=$ def. wlp
$(\mathsf{wlp}\ (v := expr)\ P)\ \wedge\ (\mathsf{wlp}\ (v := expr)\ Q$

(2) For the ";" case:

$\mathsf{wlp}\ (stmt_1; stmt_2)\ (P \wedge Q)$
$=$ def. wlp
$\mathsf{wlp}\ stmt_1\ (\mathsf{wlp}\ stmt_2\ (P \wedge Q))$
$=$ using induction hypothesis on $stmt_2$
$\mathsf{wlp}\ stmt_1\ ((\mathsf{wlp}\ stmt_2\ P) \wedge (\mathsf{wlp}\ stmt_2\ Q))$
$=$ using induction hypothesis on $stmt_1$
$(\mathsf{wlp}\ stmt_1\ (\mathsf{wlp}\ stmt_2\ P))\ \wedge\ (\mathsf{wlp}\ stmt_1\ (\mathsf{wlp}\ stmt_2\ Q))$
$=$ def. wlp
$(\mathsf{wlp}\ (stmt_1; stmt_2)\ P)\ \wedge\ (\mathsf{wlp}\ (stmt_1; stmt_2)\ Q)$

As an additional note, distributivity over $\wedge$ implies monotonicity. Imagine a function F which is distributive over $\wedge$. Suppose $P \Rightarrow Q$ is valid:

$F\ P$
$=$ P implies Q, so $P = P \wedge Q$
$F\ (P \wedge Q)$
$=$
F is distributive over $\wedge$
$F\ P\ \wedge\ F\ Q$
$\Rightarrow$
$F\ Q$